



MPI 编程实验报告

专 业： _____ 计算机科学与技术 _____
学 号： _____ 2413633 _____
班 号： _____ XXXX _____
姓 名： _____ 焦宇堃 _____

2026 年 6 月 9 日

目录

1	引言	1
1.1	多进程概念	1
2	实验配置	1
3	第一部分的实现:	2
3.1	7.1.1: 主从和任务池	2
3.2	7.1.2: 切分思路:	4
3.3	完成手册 7.1 之后测试的数据:	5
3.4	为什么没加速: 问题不只在通信, 更在任务供给	6
3.5	和上次多线程实验的关系以及回答指导书中的 7.3	8
4	两种 MPI 尝试	10
5	总结	15
A	AI 使用说明附录	15
A.1	1. 一开始关心的问题	15
A.2	2. 最后定下来的时间口径	16
A.3	3. 为什么最后选择以关键路径为主	16
A.4	4. 这次实际用到的 MPI 时间统计思路	17
A.5	5. AI 在这一部分真正帮到我的地方	17

1 引言

这次实验其实可以看成上一次多线程实验的一个延续。上一次我们主要做的是如何通过多线程去并行化 GKH 迭代。在学习了 MPI 的相关知识之后，这一次的实验的并行层次提高了一层来到了进程，也就是如果把一个程序拆成多个进程应该如何并行。

1.1 多进程概念

我们知道同一个进程里的多个线程共享大部分地址空间和资源，创建和切换通常更轻量，但也更容易遇到共享数据同步、锁竞争等问题。但进程则不同，进程都有自己的地址空间、自己的资源上下文和自己的执行流。简单说，多进程，就是让操作系统同时运行多个彼此独立的进程

我们在这次实验中使用的 MPI (Message Passing Interface) 提供的通信接口，比如点对点收发、广播、归约、同步等，让我们可以把一个并行程序拆成多个 rank，再显式地控制这些 rank 之间如何交换数据、如何分配任务、以及如何汇总结果。同样处理的函数与之前的多线程实验相同，主要都是在探索对 *gkh_svd_from_bidiagonal* 这个函数的并行可能性。这次相比于上次的线程实验，因为进程之间独立空间的特性，需要额外注意进程通信、数据打包与回收这些问题。

2 实验配置

本次实验用到的平台与之前相同，配置如下（这是一个节点的配置）：

类别	配置详情
平台系统	openEuler Linux (ARM64 架构)
常用命令	mpicc、mpic++、mpiexec
物理核心数	8 核 (Kunpeng-920)
逻辑线程数	8 线程 (不支持超线程)

表 1 并行计算平台配置信息一览

我们的测试脚本如下：

```

1 #!/bin/sh
2 #PBS -N qsub_mpi
3 #PBS -e test.e
4 #PBS -o test.o
5 #PBS -l nodes=2:ppn=8
6
7 NODES=$(cat $PBS_NODEFILE | sort | uniq)
8 # 注意把所有的ntt换成你的选题
9

```

```

10 for node in $NODEN; do
11     scp master_ubss1:/home/${USER}/svd/7.1-time ${node}:/home/${USER}
        1>&2
12     scp -r master_ubss1:/home/${USER}/svd/files ${node}:/home/${USER}
        }/ 1>&2
13 done
14
15 /usr/local/bin/mpirun -np 8 -machinefile $PBS_NODEFILE /home/${USER}
        }/7.1-time
16
17 scp -r /home/${USER}/files/ master_ubss1:/home/${USER}/svd/ 2>&1

```

3 第一部分的实现:

根据实验手册,这一部分我完成了 7.1.1 和 7.1.2 的实验,下面分别进行展示:

3.1 7.1.1: 主从和任务池

这一小节我主要完成的是把原来串行程序里的块处理过程,变成主从式进行任务分发。也就是当程序已经切出了若干个还需要继续处理的非 1x1 活动块之后,能够通过主从把任务块分给子进程进行计算,从而提高效率。

具体的实现思路上我进行如下的划分:主进程负责统一管理当前的任务池,决定哪些块需要被继续处理,并在合适的时候把任务发出去;其他进程作为从进程,在空闲时领取任务,对局部块进行计算,然后把结果返回给主进程。

在具体实现这一部分主从任务池时,我主要用到了下面几个自己写的函数:

1. send_task_to_worker

```

1 static void send_task_to_worker(const Matrix &B, const Block &blk,int
        worker, MPI_Comm comm, TimingStats &stats)
2 {
3     Matrix localB = extract_square_block(B, blk.l, blk.r);
4     std::vector<double> buf = pack_matrix(localB);
5
6     const int hdr[3] = {1, blk.l, blk.r};
7     MPI_Send(const_cast<int*>(hdr), 3, MPI_INT, worker, TAG_TASK_HDR
        , comm);
8     if (!buf.empty())
9     {
10         MPI_Send(buf.data(), static_cast<int>(buf.size()),
11                 MPI_DOUBLE, worker, TAG_TASK_DATA, comm);
12     }
13 }

```

这里做的事情就是先从全局矩阵 B 里抽出当前任务对应的局部块，再把它打包成数组，通过 MPI_Send 发给指定 worker。

2. recv_task_from_root

与发送任务对应，就是使用 MPI_Recv 一下并拿到数据

```

1 static bool recv_task_from_root(Block &blk, Matrix &localB, MPI_Comm
  comm, TimingStats &stats)
2 {
3     int hdr[3];
4     MPI_Recv(hdr, 3, MPI_INT, 0, TAG_TASK_HDR, comm,
        MPI_STATUS_IGNORE);
5     ...
6     unpack_matrix(localB, buf);
7     return true;
8 }

```

3. 回收过程:

send_result_to_root(...) 和 recv_result_from_any_worker(...) 和之前传输对应，反了一下变成把结果回传给主进程，由于篇幅原因这里不展开，核心写法同上。

因此就可以写出主从的核心部分代码:

```

1 if (rank == 0)
2 {
3     std::deque<Block> task_pool;
4     cleanup_bidiagonal(B, tol);
5     std::vector<Block> init_blocks = split_active_blocks(B, n, tol);
6     for (const auto &blk : init_blocks)
7         if (blk.r > blk.l) task_pool.push_back(blk);
8
9     for (int worker = 1; worker < size && !task_pool.empty(); ++
        worker)
10         send_task_to_worker(B, task_pool.front(), worker,
            MPI_COMM_WORLD, stats);
11
12     while (active_workers > 0)
13     {
14         TaskResult result = recv_result_from_any_worker(
            MPI_COMM_WORLD, stats);
15         insert_square_block(B, result.block_matrix, result.parent.l);
16         apply_row_rotations_local(Ut, result.u_ops);
17         apply_row_rotations_local(Vt, result.v_ops);
18
19         for (const auto &child : result.child_blocks)

```

```

20         if (child.r > child.l) task_pool.push_back(...);
21
22         if (!task_pool.empty()) send_task_to_worker(...);
23     }
24 }
25 else
26 {
27     while (true)
28     {
29         if (!recv_task_from_root(task, localB, MPI_COMM_WORLD, stats)
30             ) break;
31         process_block_until_split(localB, tol, u_ops, v_ops,
32             child_blocks);
33         send_result_to_root(task, localB, child_blocks, u_ops, v_ops,
34             MPI_COMM_WORLD, stats);
35     }
36 }

```

这部分的实验数据放到下一节实现完新的切分思路之后进行展示。

3.2 7.1.2: 切分思路:

主从框架我们已经实现了,但是就像指导手册里说的如果还按原来每一轮 iteration 把所有块扫一遍的写法走,很难把任务切细。

原因和上次我们在线程报告里说的一样,gkh_svd_from_bidiagonal 的外层 iteration 之间有很强的前后依赖。第 $k+1$ 轮到底会切出哪些块,取决于第 k 轮把 B 更新成了什么样。所以外层轮次本身不能直接并行。既然外层不能并行,那就只能对函数内部尝试并行,改成:拿住一个非 1×1 的活动块,持续推进,直到它再次分裂或者直接收敛。

在实现这部分的思路时,我这里同样用到了几个自己写的函数。这里挑最核心 process_block_until_split(...) 进行展示:

```

1 static void process_block_until_split(Matrix &localB, double tol, std
  ::vector<RowRotation> &u_ops, std::vector<RowRotation> &v_ops, std::
  vector<Block> &child_blocks)
2 {
3     while (true)
4     {
5         cleanup_bidiagonal(localB, tol);
6         handle_diagonal_zeros_emit_ops(localB, tol, u_pre, v_pre);
7         child_blocks = split_active_blocks(localB, n, tol);
8         int non_singleton_count = 0;
9         for (const auto &blk : child_blocks)
10             if (blk.r > blk.l) ++non_singleton_count;

```

```

11     if (non_singleton_count == 0 || child_blocks.size() > 1)
12         return;
13     one_block_step_emit_ops(localB, child_blocks[0].l,
14                             child_blocks[0].r, u_step, v_step);
15 }

```

这段代码先把当前局部块里的数值噪声清掉，再处理主对角线接近 0 的特殊情况，然后重新切块，看这个块现在是不是已经裂开了。如果已经裂成多个子块，或者已经收敛成 1x1，那这一次任务就可以直接结束；如果还只有一个非平凡块，那就继续往下追一步，再回到开头重新判断。

3.3 完成手册 7.1 之后测试的数据：

代码改完之后，我又专门对 np=2,4,8 做了重复实验。这里我们先主要看两类时间：

1. time gkh iteration(ms)，也就是 root 端看到的 GKH 关键路径时间；
2. 调度器里的 Time Use，也就是整次作业累计用了多少 CPU 时间。

对 1000x1000 这个大矩阵，三组数据如表 2 所示。

表 2 7.1 版本在 1000x1000 下的重复实验结果

np	GKH 第 1 次/ms	GKH 第 2 次/ms	GKH 第 3 次/ms	GKH 均值/ms	qsub 后服务器反馈的 Time Use/s
2	12958.7	11535.4	12404.0	12299.4	45 / 45 / 48
4	14388.5	11628.1	11703.4	12573.3	105 / 95 / 103
8	12974.8	11560.1	11181.6	11905.5	206 / 218 / 211

这张表其实已经能说明问题了。单看 GKH 关键路径时间，np 从 2 提高到 4、8 之后，并没有出现一种稳定、明显的加速趋势。尤其是 np=4 这一组，均值反而比 np=2 更慢。虽然 np=8 的均值略低一点，但幅度很小，和这批实验本身的波动放在一起看，不能说明它已经形成了可靠的并行收益。

更关键的是，调度器记录的 Time Use 却是一路往上走的：np=2 大约是 46 秒，np=4 大约是 101 秒，np=8 大约是 212 秒。也就是说，总资源消耗明显变大了，但我们 rank=0 的时间竟然并没有跟着明显下降。这就说明现在这套并行方式，至少从结果上看，确实还没有把多进程并行计算有效转化成明显加速。

45595.master_ubss1	qsub_mpi	s2413633	00:00:45 C deque
45598.master_ubss1	qsub_mpi	s2413633	00:00:45 C deque
45600.master_ubss1	qsub_mpi	s2413633	00:00:48 C deque
45604.master_ubss1	qsub_mpi	s2413633	00:01:45 C deque
45607.master_ubss1	qsub_mpi	s2413633	00:01:35 C deque
45609.master_ubss1	qsub_mpi	s2413633	00:01:43 C deque
45614.master_ubss1	qsub_mpi	s2413633	00:03:26 C deque
45622.master_ubss1	qsub_mpi	s2413633	00:03:38 C deque
45625.master_ubss1	qsub_mpi	s2413633	00:03:31 C deque

图 1 7.1 重复实验原始时间结果

```

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 3311.61
总GKH迭代耗时(ms): 12960.3
通过: 5 / 5

```

图 2 np=2 测试实验中的一次截图

```

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 5515.44
总GKH迭代耗时(ms): 14390.2
通过: 5 / 5

```

图 3 np=4 测试实验中的一次截图

```

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 5660.67
总GKH迭代耗时(ms): 12976.6
通过: 5 / 5

```

图 4 np=8 测试实验中的一次截图

3.4 为什么没加速：问题不只在通信，更在任务供给

如果只看表 2，我们最多只能说“没加速”，因此我通过对代码的修改统计了一些时间参数：

1. `time gkh iteration(ms)`: 统计的是 GKH 这一段主计算过程在 root 端看到的关键路径时间，也就是这一部分真正花了多久。
2. `MPI_MAX wall`: 统计的是一次并行执行里最慢那个进程的墙钟时间（大部分时间都是 rank=0 的主进程），更接近整轮任务实际完成所需要等待的时间。
3. `comm subtotal`: 统计的是收发任务、等待结果这些通信相关时间的总和，用来判断通信成本是不是已经比较重了。
4. `task wait` 和 `result wait`: 前者统计等新任务或等任务返回的时间，后者统计结果回收这一侧的等待时间，用来判断是不是有很多进程其实是在空等。
5. `compute`: 统计的是真正落在块计算上的时间，也就是实际做数值更新花掉的时间。
6. `merge`: 统计的是主进程把结果收回来之后，重新合并到全局矩阵并应用旋转的时间。
7. `Time Use`: 这是调度器记录的累计 CPU 使用时间，更像总资源的分配。

然后我们对 7.1 版本打印出来的 profiling 进行分析 (这里展示的是三次重复的平均值整理后的相同随机数种子 1000x1000 样例):

表 3 7.1 版本在 1000x1000 下的平均时间分解

np	MPI_MAX	wall/ms	comm	subtotal/ms	task wait/ms	result wait/ms	compute/ms	merge/ms
2		12464.1		13153.5	5195.1	5723.8	4586.6	2719.8
4		13319.4		38210.0	30566.0	5552.6	4390.7	3333.2
8		12955.8		83857.1	76653.9	5593.2	4444.6	3074.1

这个表能看出两个很明显的现象。

第一, 真正的 compute 时间几乎没有随着 np 增大而同步下降, 基本都还在 4.4 ~ 4.6 秒这个量级。这说明新增进程并没有把实打实的块计算量均匀分走。

第二, task wait 和整体 comm subtotal 却涨得非常快。np=2 时平均 task wait 只有 5.2 秒; 到 np=4 时已经到了 30.6 秒; np=8 时更是到了 76.7 秒。这个趋势非常说明问题: 进程确实开多了, 但是很多时候它们不是在算, 而是在等。

所以这里不能简单地说可能是拷贝、通信带来的时间增加, 应该是:

1. 主从框架和打包收发本身会带来额外开销;
2. 但更核心的问题是, 任务池并没有长期提供足够多的独立任务去喂满这些 worker;
3. 在任务供给不足的前提下, 进程数一加大, 等待、收发、合并这些成本就会被放大出来。

因此通过以上的分析, 我们可以认为目前版本问题并不是单纯某一次通信太慢, 而是整个高层任务组织方式本身就很难把这些进程持续喂满。

```
[7.1 master-worker timing]
total wall(ms, MPI_MAX) : 12958.2
task prepare(ms) : sum 1333.79, max 1333.79
task send(ms) : sum 2271.06, max 2271.06
task wait recv(ms) : sum 4674.83, max 4674.83
task unpack(ms) : sum 348.327, max 348.327
compute(ms) : sum 4639.15, max 4639.15
result pack(ms) : sum 430.658, max 430.658
result send(ms) : sum 937.339, max 937.339
result wait recv(ms) : sum 5753.68, max 5753.68
result unpack(ms) : sum 422.675, max 422.675
merge+apply(ms) : sum 2494.39, max 2494.39
finalize(ms) : sum 49.9358, max 49.9358
stop send(ms) : sum 0.000476837, max 0.000476837
comm subtotal(ms, sum) : 13636.9
serdes subtotal(ms, sum) : 2535.45
tasks dispatched/executed : 994 / 994
payload bytes task/result : 2668600304 / 2705831448
converged : yes
||A-U*S*V^T||_F : 2.06478e-10
relative recon error : 3.5722e-13
||U^T U-I||_F : 1.96832e-13
||V^T V-I||_F : 1.97124e-13
diagonal structure error : 0
descending order error : 0
nonnegative diagonal : yes
time bidiagonalization(ms): 3311.55
time gkh iteration(ms) : 12958.7
结果: PASS
```

图 5 np=2 测试实验中的一次 time

```
[7.1 master-worker timing]
total wall(ms, MPI_MAX) : 14387.8
task prepare(ms) : sum 1468.73, max 1468.73
task send(ms) : sum 1953.16, max 1953.16
task wait recv(ms) : sum 33268.2, max 14294.6
task unpack(ms) : sum 442.134, max 442.134
compute(ms) : sum 4441.42, max 4441.42
result pack(ms) : sum 440.645, max 440.645
result send(ms) : sum 1044.12, max 1044.12
result wait recv(ms) : sum 5687.34, max 5687.34
result unpack(ms) : sum 428.858, max 428.858
merge+apply(ms) : sum 4091.63, max 4091.63
finalize(ms) : sum 46.4351, max 46.4351
stop send(ms) : sum 0.00143051, max 0.00143051
comm subtotal(ms, sum) : 41952.8
serdes subtotal(ms, sum) : 2780.37
tasks dispatched/executed : 994 / 994
payload bytes task/result : 2668600304 / 2705831448
converged : yes
||A-U*S*V^T||_F : 2.06478e-10
relative recon error : 3.5722e-13
||U^T U-I||_F : 1.96832e-13
||V^T V-I||_F : 1.97124e-13
diagonal structure error : 0
descending order error : 0
nonnegative diagonal : yes
time bidiagonalization(ms): 5515.39
time gkh iteration(ms) : 14388.5
结果: PASS
```

图 6 np=4 测试实验中的一次 time

```
[7.1 master-worker timing]
total wall(ms, MPI_MAX) : 12974.1
task prepare(ms) : sum 1412.87, max 1412.87
task send(ms) : sum 603.204, max 603.204
task wait recv(ms) : sum 73916.2, max 12429.2
task unpack(ms) : sum 426.614, max 426.614
compute(ms) : sum 4565.91, max 4565.91
result pack(ms) : sum 458.171, max 458.171
result send(ms) : sum 1066.97, max 1066.97
result wait recv(ms) : sum 5775.9, max 5775.9
result unpack(ms) : sum 416.183, max 416.183
merge+apply(ms) : sum 3936.94, max 3936.94
finalize(ms) : sum 56.9391, max 56.9391
stop send(ms) : sum 0.00405312, max 0.00405312
comm subtotal(ms, sum) : 81362.2
serdes subtotal(ms, sum) : 2713.84
tasks dispatched/executed : 994 / 994
payload bytes task/result : 2668600304 / 2705831448
converged : yes
||A-U*S*V^T||_F : 2.06478e-10
relative recon error : 3.5722e-13
||U^T U-I||_F : 1.96832e-13
||V^T V-I||_F : 1.97124e-13
diagonal structure error : 0
descending order error : 0
nonnegative diagonal : yes
time bidiagonalization(ms): 5660.63
time gkh iteration(ms) : 12974.8
结果: PASS
```

图 7 np=8 测试实验中的一次 time

3.5 和上次多线程实验的关系以及回答指导书中的 7.3

这一点其实正好能和上次的多线程实验接上。上次我已经感觉问题可能不只是线程不够多，而是高层块任务本身就不太容易并行。到了这次 MPI 实验，根据指导手册里的要求我换了多组随机种子，把每一轮的块信息打出来，想确认到底是不是这里出了问题。打印代码如下：

```
1 cleanup_bidiagonal(B, tol);
2 handle_diagonal_zeros_emit_ops(B, tol, u_ops, v_ops);
3
4 std::vector<Block> tmp_blocks = split_active_blocks(B, n, tol);
5 blocks.swap(tmp_blocks);
```


行框架搭出来以后，真正卡住它的，不是“没有任务池”，而是“任务池里大多数时候根本没有足够多的非平凡块”。

因此我们就可以串起来我们的实验工作：

1. 7.1.1 负责把主从和任务池搭起来；
2. 7.1.2 负责把任务推进粒度改成按块推进；
3. 7.3 则说明，这个任务池在绝大多数时候其实只能拿到 1 个真正可并行的非 1×1 块。因此，新增进程没有办法长期拿到有效工作量，最后看到的就是资源用得更多，但时间并没有明显变快，这可以作为原因反过来解释 7.1 的工作结果差的原因。

4 两种 MPI 尝试

在把 7.1.1 和 7.1.2 做完之后，我后面又继续尝试了两种不同的 MPI 写法。它们都还是围绕同一个 `gkh_svd_from_bidiagonal` 展开，但在具体的进程职能划分上，思路并不一样。这里我把它分别记成 MPI1 和 MPI2。

还需要说明的是我在多进程中对 `main` 进行了微小调整，只有 `rank=0` 的会进行打印输出，这是因为其他的进程都是 `worker`，拿到的都是不完整的数据，因此如果对这部分也进行校验打印出来的必定是 `fail`。

MPI1：固定角色的三进程

第一种尝试的思路比较直接，就是把三个进程的角色固定下来：

1. `rank 0` 负责维护 B ，并在 `root` 上推进块任务；
2. `rank 1` 专门维护 U^T ；
3. `rank 2` 专门维护 V^T ；
4. 每次 `root` 处理完一个块之后，再把这一步产生的 `u_ops` 和 `v_ops` 分别发给另外两个进程。

这一版我这里同样不把所有辅助函数都展开，而是只放一段最能说明思路的主干代码：

```

1 if (rank == 0)
2 {
3     Matrix Ut = transpose_copy(U);
4     Matrix Vt = transpose_copy(V);
5     send_matrix_to_rank(Ut, 1, TAG_UT_INIT, MPI_COMM_WORLD);
6     send_matrix_to_rank(Vt, 2, TAG_VT_INIT, MPI_COMM_WORLD);
7
8     while (!task_pool.empty() && task_budget < max_iter)
9     {

```

```

10     Matrix localB = extract_square_block(B, task.l, task.r);
11     process_block_until_split(localB, tol, u_ops, v_ops,
12                               child_blocks);
13     insert_square_block(B, localB, task.l);
14
15     MPI_Send(&ctrl, 1, MPI_INT, 1, TAG_CTRL, MPI_COMM_WORLD);
16     MPI_Send(&ctrl, 1, MPI_INT, 2, TAG_CTRL, MPI_COMM_WORLD);
17     send_row_rotations_to_rank(u_ops, 1, TAG_U_TASK,
18                               MPI_COMM_WORLD);
19     send_row_rotations_to_rank(v_ops, 2, TAG_V_TASK,
20                               MPI_COMM_WORLD);
21 }
22 }
23 else if (rank == 1)
24 {
25     Matrix Ut = recv_matrix_from_rank(m, m, 0, TAG_UT_INIT,
26                                       MPI_COMM_WORLD);
27     apply_row_rotations_local(Ut, u_ops);
28 }
29 else if (rank == 2)
30 {
31     Matrix Vt = recv_matrix_from_rank(n, n, 0, TAG_VT_INIT,
32                                       MPI_COMM_WORLD);
33     apply_row_rotations_local(Vt, v_ops);
34 }

```

这段代码就是先把 U^T 和 V^T 分别交给两个固定进程保管，然后 root 每处理完一个块，就把这一轮产生的旋转操作分别发出去，让另外两个进程各自更新自己的矩阵。

这一版的优点是角色很清楚，但缺点也很明显，就是这种方式把并行结构固定死在了 np=3 上，而且 root 仍然承担了主要的块推进工作，而不是一个可以自然扩展到更多进程的通用版本。

表 5 MPI1 在 1000x1000 下的 profiling 均值（5 次重复）

进程数	通过次数	GKH 均值/ms	关键路径 Wall/ms	Root 处理/ms	Root 抽取回填/ms	通信累计/ms	更新累计/ms	Time Use/s
3	5 / 5	5851.45	6051.10	4455.22	927.35	9904.09	1842.37	48.4

这里我统计的都是大矩阵 1000x1000 这一档，因为真正能拉开差距的主要就是这一组。从这张表里可以看到，MPI1 这一版是能稳定跑通的，5 次重复都是 5/5 PASS。它的 rank=0 主进程 GKH 时间均值是 5851.45 ms，我们相比于 7.1 的版本进行了加速。表里的“关键路径 Wall”统计的是 MPI_MAX wall，也就是整轮执行真正决定总耗时的那条关键路径；而“通信累计”和“更新累计”统计的是把所有进程上的对应时间全部加起来之后的

结果，所以它们不能直接和 GKH 或 Wall 去逐项相加。因此我们可以看到，rank=0 自己处理块的时间均值已经到了 4455.22 ms，rank=0 抽取和回填块又花了 927.35 ms，说明主要工作还是压在 rank=0 上；与此同时，三进程合起来累计在通信上又花了 9904.09 ms，说明这套固定角色方案在全体进程上的通信成本也并不低。也就是说，这一版并行结构依旧是还是很偏向 rank=0 的，另外两个进程是在辅助计算。

```
-----
45777.master_ubss1      qsub_mpi      s2413633      00:00:46 C dque
45779.master_ubss1      qsub_mpi      s2413633      00:00:49 C dque
45780.master_ubss1      qsub_mpi      s2413633      00:00:51 C dque
45781.master_ubss1      qsub_mpi      s2413633      00:00:49 C dque
45782.master_ubss1      qsub_mpi      s2413633      00:00:47 C dque
-----
```

图 9 固定角色的结果

```
diagonal structure error : 0
descending order error   : 0
nonnegative diagonal     : yes
time bidiagonalization(ms): 4891.06
time gkh iteration(ms)   : 5729.53
结果: PASS

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 4891.11
总GKH迭代耗时(ms): 5731.17
通过: 5 / 5
```

图 10 固定角色的结果二（局部截图）

MPI2：列块分发加广播同步的通用方案

这一版我当时想解决的核心问题是：既然第一种写法把进程角色固定死了，那能不能改成一个更一般的方案，让所有进程都真正参与到 U^T 和 V^T 的更新里来。

这里还有一个思路上的取舍需要先说清楚。按直觉看，最想并行的当然还是 B 本身，因为真正的 GKH 推进主要都发生在 B 上。但问题在于， B 这一侧的块推进有很强的连续性：当前这一轮怎么切块、下一步追到哪里、块什么时候重新分裂，这些都强依赖前一步刚刚算出来的结果，所以它并不适合像普通数据并行那样，直接把 B 很自然地切给多个进程同时独立推进。正因为这一层不好直接做进程化，我这里才换了一个角度，去尝试把 U^T 和 V^T 的更新拆给多个进程，让主进程继续维护 B ，而其他进程去并行承担旋转应用这一部分。

但这样做也有一个很自然的预期： U^T 和 V^T 的更新虽然可以拆开，但它们本身并不是这整个流程里最重的那一部分。如果为了让所有进程都参与进来，反而要额外付出列块分发、广播旋转、最终 gather 回收这些成本，那么最后很可能出现的情况就是：通信和拷贝的代价比真正省下来的计算还大。所以从思路上说，MPI2 这版虽然更通用，但有可能因为同步和通信更重，最后反而更慢。

所以这一次的思路变成了：

1. 先把 U^T 和 V^T 按列块切开，分发给所有进程；
2. root 还是负责维护 B 并推进块任务；
3. 每处理完一个块，root 把这一轮生成的旋转操作广播给所有进程；
4. 每个进程只在自己手里的局部列块上应用这些旋转；
5. 最后再把各自的列块 gather 回 root，拼成完整的 U^T 和 V^T 。

主干代码如下：

```

1  if (rank == 0)
2  {
3      Ut_local = scatter_col_blocks_from_root(Ut_full, m, m, 0,
4          MPI_COMM_WORLD);
5      Vt_local = scatter_col_blocks_from_root(Vt_full, n, n, 0,
6          MPI_COMM_WORLD);
7  }
8  else
9  {
10     Ut_local = scatter_col_blocks_from_root(dummy_u, m, m, 0,
11         MPI_COMM_WORLD);
12     Vt_local = scatter_col_blocks_from_root(dummy_v, n, n, 0,
13         MPI_COMM_WORLD);
14 }
15
16 while (true)
17 {
18     if (rank == 0)
19     {
20         Matrix localB = extract_square_block(B, task.l, task.r);
21         process_block_until_split(localB, tol, u_ops, v_ops,
22             child_blocks);
23         insert_square_block(B, localB, task.l);
24     }
25
26     MPI_Bcast(&control, 1, MPI_INT, 0, MPI_COMM_WORLD);
27     bcast_row_rotations(u_ops, 0, MPI_COMM_WORLD);
28     bcast_row_rotations(v_ops, 0, MPI_COMM_WORLD);
29     apply_row_rotations_local(Ut_local, u_ops);
30     apply_row_rotations_local(Vt_local, v_ops);
31 }

```

```

28 Matrix Ut_gathered = gather_col_blocks_to_root(Ut_local, m, m, 0,
    MPI_COMM_WORLD);
29 Matrix Vt_gathered = gather_col_blocks_to_root(Vt_local, n, n, 0,
    MPI_COMM_WORLD);

```

这段代码的意思是：先把完整矩阵拆成每个进程各管一块，然后由 root 统一广播每一步产生的旋转操作，所有进程再同步更新自己手里的局部列块。

这部分的时间统计如下：

表 6 MPI2 在 1000x1000 下的 profiling 均值（分别三次平均值）

进程数	通过次数	GKH 均值/ms	关键路径 Wall/ms	主进程处理/ms	主进程抽取回填/ms	通信累计/ms	更新累计/ms	Time Use/s
2	3 / 3	7896.71	7896.19	5082.59	1007.04	6746.12	2210.70	35.3
4	3 / 3	7694.35	7931.81	5218.42	1056.86	20983.20	2235.08	80.3
8	3 / 3	7166.82	8005.29	4903.31	983.93	49006.93	2162.94	177.3

首先这一版在 np=2,4,8 下都能稳定跑通，三次重复也都是 5/5 PASS。再看时间，time gkh iteration(ms) 的均值确实有一点下降，从 np=2 的 7896.71 ms 降到了 np=8 的 7166.82 ms。但如果把 MPI_MAX wall 和 Time Use 放在一起看，问题就出来了：墙钟时间并没有跟着明显下降，基本一直在 7.9 ~ 8.0 s 这一带，而 Time Use 却从 35.3 s 一路涨到了 177.3 s。

更关键的是通信项。comm subtotal 从 np=2 的 6746.12 ms 直接涨到 np=8 的 49006.93 ms，增长非常明显。因此我们可以分析出随着进程数增加，有效计算没有随之变多，广播、同步和回收这些成本也符合我们预期上升的比较明显。

所以 MPI2 这一版虽然结构更通用，但是在当前 SVD 这类任务下，进程一加多，通信负担会被非常快地放大出来。

```

==2
45784.master_ubss1      qsub_mpi      s2413633      00:00:34 C deque
45785.master_ubss1      qsub_mpi      s2413633      00:00:33 C deque
45787.master_ubss1      qsub_mpi      s2413633      00:00:39 C deque

```

图 11 MPI2: np=2 的结果

```

结果: PASS
=====
随机种子基值: 20260408
总上二对角化耗时(ms): 3724.16
总GKH迭代耗时(ms): 7844.42
通过: 5 / 5

```

图 12 MPI2: np=2 的结果二（局部截图）

```

== 4
45789.master_ubss1      qsub_mpi      s2413633      00:01:23 C deque
45790.master_ubss1      qsub_mpi      s2413633      00:01:18 C deque
45791.master_ubss1      qsub_mpi      s2413633      00:01:20 C deque

```

图 13 MPI2: np=4 的结果


```

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 5439
总GKH迭代耗时(ms): 7581.83
通过: 5 / 5
Authorized users only. All activities may be monitored and reported.

```

图 14 MPI2: np=4 的结果二（局部截图）

```

=====
45792.master_ubss1      qsub_mpi      s2413633      00:02:55 C dque
45793.master_ubss1      qsub_mpi      s2413633      00:03:03 C dque
45794.master_ubss1      qsub_mpi      s2413633      00:02:54 C dque

```

图 15 MPI2: np=8 的结果

```

=====
随机种子基值: 20260408
总上二对角化耗时(ms): 6944.78
总GKH迭代耗时(ms): 7134.13
通过: 5 / 5
Authorized users only. All activities may be monitored and reported.

```

图 16 MPI2: np=8 的结果二（局部截图）

5 总结

这次实验我们围绕 SVD 的 GKH 迭代，连续尝试了几条不同层次的多进程并行化。最开始做的是把原来按外层迭代整体推进的写法，改成按非 1×1 活动块推进，并在这个基础上搭出主从和任务池框架。之后，我们又继续尝试了两种 MPI 写法：一种是固定角色的三进程方案，由主进程维护 B ，另外两个进程分别维护 U^T 和 V^T ；另一种是更通用的列块分发加广播同步方案，让更多进程一起参与 U^T 和 V^T 的更新。

从结果上看，首先，7.1 这一层说明主从和任务池是能搭起来的，程序也能稳定跑通，但提高进程数以后并没有得到明显加速。接着，再结合日志统计可以看到，绝大多数时候实际上只有一个非 1×1 的活动块在继续推进，这就意味着高层任务池里长期没有足够多的独立任务去喂满这些进程；之后 MPI 中尝试固定三角色的方案结果相对更快；更通用的广播同步方案虽然能自然扩展到更多进程，但随着进程数增加，通信、同步和数据分发的开销涨得非常快。

如果说这次实验最大的收获，是我对 profiling 有了更深层的理解，因为很多现象只看总时间是看不出来的，必须把关键路径、通信累计、等待、局部更新这些细节子量拆开，才能知道问题到底卡在哪。

A AI 使用说明附录

这一部分主要记录我在实验分析阶段，如何借助 AI 一起把 profiling 的口径一步一步定下来。这里我想保留的不是简单一句“用了 AI”，而是把我当时真正关心的问题和最终形成的分析思路交代清楚。

A.1 1. 一开始关心的问题

我在后面开始写 profiling 的时候，最先关心的其实不是“表格怎么画”，而是“到底应该统计哪些时间”。因为这次实验里有主进程、有子进程、有点对点通信、有广播同步，

还有一些看起来很像时间、但含义其实不一样的量。如果这里口径没定好，后面所有分析都会很乱。

当时我主要围绕下面几个问题和 AI 讨论：

1. 我们最后到底应该看主进程的时间，还是看所有进程的总时间。
2. 如果想说明程序真正有没有变快，关键路径应该怎么统计。
3. MPI 里有哪些现成 API 可以用来做时间统计。
4. 哪些量适合拿来说明“用户实际等了多久”，哪些量适合拿来说明“总资源消耗有多大”。
5. 两个 MPI 版本的 profiling 口径不完全一样时，后面该怎么对齐分析。

A.2 2. 最后定下来的时间口径

最后我没有把所有时间都混在一起，而是把它们拆成了几层来看。

1. 第一层是关键路径时间。这里我主要看 `time gkh iteration(ms)` 和 `MPI_MAX wall`。因为这两个量更接近“这轮程序真正跑完，用户等了多久”。
2. 第二层是主进程本地工作量。这里我重点看 `root process block` 和 `root extract+insert`，因为它们能直接反映 root 自己是不是已经很重了。
3. 第三层是通信和同步负担。这里我看的是 `comm subtotal`，也就是把所有进程上的通信时间累计起来之后得到的值。它不是关键路径时间，但可以说明总体通信负担是不是在变重。
4. 第四层是本地更新量。这里看的是 `local update`，也就是各个进程在本地应用旋转、更新矩阵的累计时间。
5. 第五层是调度器的 Time Use。这个量不代表关键路径，但它能说明整次作业到底吃掉了多少 CPU 资源，所以特别适合用来说明“资源用得更多了，但程序不一定真的更快”。

也就是说，我最后并不是只看一个总时间，而是把“关键路径”“主进程负担”“通信负担”“本地更新”“总资源消耗”这几层分开来看。这样做的好处是，即使程序没有加速，我们也能知道它到底是卡在 root、卡在通信，还是卡在任务供给上。

A.3 3. 为什么最后选择以关键路径为主

我后面和 AI 讨论下来，最重要的一个收获其实就是：并行程序分析时，不能把所有时间都按同一个层次去理解。

比如一开始看到 `comm subtotal` 比 `time gkh iteration(ms)` 还大时，我也会觉得这个数是不是有问题。后来才慢慢理清楚，这两个量本身就不是一个层次：

1. `time gkh iteration(ms)` 和 `MPI_MAX wall` 反映的是关键路径，也就是用户真正等了多久。
2. `comm subtotal` 和 `local update` 是把多个进程上的时间做累计之后得到的，所以它们本来就可以比关键路径更大。

把这件事想清楚之后，我后面的表头和解释也跟着改了。也就是后来报告里写的“关键路径 Wall”“通信累计”“更新累计”这些说法，本质上就是在避免把不同层次的时间混为一谈。

A.4 4. 这次实际用到的 MPI 时间统计思路

在实现上，我这次没有去用特别复杂的外部 profiler，而是优先在代码里加时间点，把程序运行过程拆开来统计。这里最常用的就是 `MPI_Wtime()`，因为它比较适合拿来做法 MPI 程序内部的分段计时。

具体来说，思路就是：

1. 在一段逻辑开始前记一个时间点；
2. 在结束后再记一个时间点；
3. 用前后差值算这一段花了多少时间；
4. 最后再根据这段时间属于“root 处理”“通信”“本地更新”中的哪一类，把它累计到对应统计量里。

像 root 处理块、收发控制消息、广播旋转、应用旋转、最终 gather 这些时间，后面基本都是按这个思路一点一点加进去的。

A.5 5. AI 在这一部分真正帮到我的地方

如果只说“AI 帮我写了点文字”，那其实太表面了。对我来说，它在这一部分真正帮到我的地方主要有三个：

1. 帮我把“应该看哪个时间”这件事先分层理清楚了，特别是关键路径时间和累计时间不能混着理解这一点。
2. 帮我把原来代码里已经有的一些统计项重新组织成可以写进报告的口径，而不是只停留在打印了一堆数字。
3. 帮我把后面表格里的解释改得更严谨一些，避免出现“这些时间看起来加不起来，所以是不是数据错了”这种误读。

所以这一部分保留下来，我觉得最大的意义不是证明“我用了 AI”，而是把我这次 profiling 的思路演化过程记录下来：一开始我关心关键路径该怎么统计，后来逐步明确了要把主进程、本地更新、通信累计和总资源消耗拆开来看，最后才形成了现在报告里这套比较稳定的分析口径。